Lab 10 Instructions

Aditya Sarkar

Tulane Dr, Hyattsville, MD | November 11, 2026

1 `wav_bytes`

1.1 Functional Purpose

Computes the exact size of an uncompressed PCM `.wav` file for a specified duration in seconds. It calculates:

- The total number of discrete samples matching that duration (rounded to the nearest whole integer).
- The data payload byte count ($N_{\text{data}} = \text{samples} \times \text{block_align}$).
- The complete file footprint by adding the RIFF format header size.

1.2 Line-by-Line Implementation Steps

1. Calculate the frame stride by invoking `ba = block_align(channels, bit_depth)`. This computes the bytes per audio frame and checks parameter validity.
2. Validate `sample_rate` by calling `_positive_int(sample_rate, "sample_rate")`.
3. Validate the parameter `seconds`:
   - If `isinstance(seconds, bool)` or not `isinstance(seconds, (int, float))`, raise: `CaptureError(f"seconds is {seconds!r}, which is not a duration.")`
   - If `float(seconds) < 0.0`, raise: `CaptureError(f"seconds is {seconds}, which is not a duration.")`
4. Validate the parameter `header_bytes`:
   - If `isinstance(header_bytes, bool)`, not an `int`, or `header_bytes < 0`, raise: `CaptureError(f"header_bytes is {header_bytes!r}.")`
5. Compute discrete sample count: Multiply `float(seconds)` by `sample_rate`, round with `round()`, and cast using `int()`.
6. Compute payload data bytes: Multiply `samples` by `ba`.
7. Return a dictionary containing:
   - `"samples"`: the calculated sample count.

- `"block_align"`: the frame stride `ba`.
- `"byte_rate"`: `sample_rate * ba`.
- `"data_bytes"`: the raw data byte count.
- `"header_bytes"`: the header size.
- `"total_bytes"`: the sum of data bytes and header bytes.

2 `duration_from_bytes`

2.1 Functional Purpose

Performs the inverse operation of file sizing: given a raw audio byte count, it determines how long the audio lasts and whether the byte count represents complete audio frames. A non-zero remainder indicates mid-write termination or file truncation.

2.2 Line-by-Line Implementation Steps

1. Determine frame stride: Call `ba = block_align(channels, bit_depth)`.
2. Validate sample rate: Run `_positive_int(sample_rate, "sample_rate")`.
3. Validate `data_bytes`:
   - If `isinstance(data_bytes, bool)`, not an `int`, or `data_bytes < 0`, raise: `CaptureError(f"data_bytes is {data_bytes!r}, which is not a byte count.")`
4. Divide data bytes by frame stride: Use `samples, remainder = divmod(data_bytes, ba)`.
5. Return a dictionary containing:
   - `"samples"`: the integer quotient `samples`.
   - `"seconds"`: duration calculated as `samples / sample_rate`.
   - `"remainder"`: leftover bytes stored in `remainder`.
   - `"whole"`: boolean flag evaluated as `remainder == 0`.

3 `header_consistency`

3.1 Functional Purpose

Cross-examines the actual binary WAV header metadata against the metadata written down in the experimental session record. It ensures the header values match the session notes for sample rate, channel count, and bit depth, and recomputes the true duration from the header's byte count.

3.2 Line-by-Line Implementation Steps

1. Ensure both `header` and `audio` are dictionaries. If either fails `isinstance(..., dict)`, raise:
   `CaptureError("header and audio must both be objects.")`
2. Initialize an empty list: `mismatches = []`.
3. Iterate over the tuple `("sample_rate", "channels", "bit_depth")`:
   - If the field is not in `header` or not in `audio`, raise:
     `CaptureError(f"{field!r} is missing from the header or the audio block; nothing can be compared.")`
   - If `header[field] != audio[field]`, append a mismatch dictionary to `mismatches`:
     `{"field": field, "header": header[field], "audio": audio[field]}`
4. Verify the presence of data payload: If `"data_bytes"` is not in `header`, raise:
   `CaptureError("the header has no 'data_bytes'.")`
5. Recompute true duration: Call `duration_from_bytes` passing `header["data_bytes"]`, `header["sample_rate"]`, `header["channels"]`, and `header["bit_depth"]`, saving to `dur`.
6. Return a dictionary containing:
   - `"agree"`: evaluated as `not mismatches`.
   - `"mismatches"`: the list of detected mismatch dictionaries.
   - `"duration"`: the `dur` dictionary calculated in step 5.
   - `"stated_seconds"`: cast value `float(audio.get("seconds", 0.0))`.

4 `dbfs`

4.1 Functional Purpose

Converts a normalized linear amplitude magnitude in $[0.0, 1.0]$ to decibels relative to full scale (dBFS).

$$\text{dBFS} = 20 \log_{10}(\text{amplitude}) \tag{1}$$

Absolute digital silence (0.0) explicitly returns `None` rather than $-\infty$ to maintain JSON serialization compatibility and force downstream modules to handle digital silence deliberately.

4.2 Line-by-Line Implementation Steps

1. Verify that `amplitude` is a valid numeric scalar:
   - If `isinstance(amplitude, bool)` or not `isinstance(amplitude, (int, float))`, raise:
     `CaptureError(f"amplitude is {amplitude!r}, which is not a number.")`
2. Cast the amplitude to floating-point: `a = float(amplitude)`.

3. Enforce positive magnitude constraints:
   - If `a < 0.0`, raise:
     `CaptureError(f"amplitude is {a}. Levels here are magnitudes in [0, 1]; a negative one is a raw sample, and the sign of a raw sample says nothing about how loud it is.")`
4. Handle digital silence: If `a == 0.0`, return `None`.
5. Return decibel value: `20.0 * math.log10(a)`.

5 `bits_in_use`

5.1 Functional Purpose

Determines the effective dynamic bit range utilized by the signal peak. If a 16-bit capture peaks at -40 dBFS, it only utilizes roughly 9.36 bits of quantizer resolution, leaving 6.64 bits unused.

5.2 Line-by-Line Implementation Steps

1. Validate `bit_depth` with `_positive_int(bit_depth, "bit_depth")`.
2. Check for silence: If `peak_dbfs is None`, return `0.0`.
3. Validate the level parameter:
   - If `isinstance(peak_dbfs, bool)` or not `isinstance(peak_dbfs, (int, float))`, raise:
     `CaptureError(f"peak_dbfs is {peak_dbfs!r}, which is not a level.")`
4. Compute effective bits: Add the negative decibel offset scaled by `DB_PER_BIT` to the format's `bit_depth`, guarding the lower boundary at zero:

$$\max(0.0, \texttt{bit_depth} + \texttt{float(peak_dbfs)}/\texttt{DB_PER_BIT})$$

6 `level_sentence`

6.1 Functional Purpose

Generates a human-readable, single-sentence summary of the level evaluation dictionary, articulating peak levels, RMS levels, dynamic crest factor, and clipping counts.

6.2 Line-by-Line Implementation Steps

1. Check for digital silence: If `level["silent"]` is true, return:
   `f"the capture is digitally silent: all {level['samples']} samples are zero"`
2. Extract peak value `p = level["peak_dbfs"]` and initialize the list:
   `parts = [f"peak {p:.2f} dBFS"]`

3. Check if `level["rms_dbfs"] is not None`: If true, append `f"RMS {level['rms_dbfs']:.2f} dBFS"` to `parts`.
4. Check if `level["crest_db"] is not None`: If true, append `f"crest {level['crest_db']:.2f} dB"` to `parts`.
5. Check if `level["clipped"]` is non-zero: If true, append `f"{level['clipped']} samples at the rail ({100.0 * level['clip_rate']:.4f}%)"` to `parts`.
6. Return comma-delimited string: `", ".join(parts)`.

7 `record_gaps`

7.1 Functional Purpose

Audits an audio session metadata dictionary against the required 13 benchmark capture decisions (`THIRTEEN_CHOICES`). It classifies parameters into:

- Populated fields.
- Completely missing keys.
- Present but empty or whitespace-only keys.
- Fields extracted directly from file headers versus handwritten operator notes.

7.2 Line-by-Line Implementation Steps

1. Confirm dictionary structure: If not `isinstance(record, dict)`, raise: `CaptureError(f"the record is a {type(record).__name__}, not an object.")`
2. Initialize tracking lists: `present = []`, `missing = []`, `blank = []`.
3. Loop through each key in `THIRTEEN_CHOICES`:
   - If `key not in record`: append to `missing`.
   - Else if `not str(record[key]).strip()`: append to `blank`.
   - Otherwise: append to `present`.
4. Return a dictionary containing:
   - `"present"`: list of present keys.
   - `"missing"`: list of missing keys.
   - `"blank"`: list of present but empty keys.
   - `"complete"`: boolean flag evaluated as `not missing and not blank`.
   - `"in_header"`: cast copy of `list(CHOICES_IN_HEADER)`.
   - `"hand_written"`: list comprehension containing keys in `THIRTEEN_CHOICES` that are not in `CHOICES_IN_HEADER`.

8 `record_sentence`

8.1 Functional Purpose

Formats the dictionary generated by `record_gaps` into an executive status string indicating whether all capture choices have been documented or listing missing parameters.

8.2 Line-by-Line Implementation Steps

1. Count present fields: `n = len(gaps["present"])`.
2. Compute total required choices: `total = n + len(gaps["missing"]) + len(gaps["blank"])`.
3. Check for full completion: If `gaps["complete"]` is true, return:
   `f"all {total} choices are named, {len(gaps['in_header'])} of them also in the file header and {len(gaps['hand_written'])} of them nowhere else"`
4. If gaps exist, initialize an empty list `parts = []`:
   - If `gaps["missing"]` is non-empty: append
     `f"{len(gaps['missing'])} missing ({', '.join(gaps['missing'])})"`.
   - If `gaps["blank"]` is non-empty: append
     `f"{len(gaps['blank'])} present and empty ({', '.join(gaps['blank'])})"`.
5. Return combined summary: `f"{n} of {total} choices are named; " + "; ".join(parts)`.